Bluetooth e Bluetooth Low Energy

Tecnologias de Comunicação para a Internet das Coisas, Aula 04

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Agenda (1/2)

- A tecnologia Bluetooth: origem, bandas de operação e classes de potência
- Bluetooth Low Energy: motivação e comparação com o Bluetooth clássico
- Camada física e camada de ligação (Link Layer)
- Anúncios, varrimento e estabelecimento de ligação
- Generic Access Profile (GAP)
- Attribute Protocol (ATT) e Generic Attribute Profile (GATT)

Agenda (2/2)

- Segurança: emparelhamento, autenticação e vinculação
- Aplicações e ferramentas de gestão
- Evolução das versões, privacidade e alternativas sem hardware (emulação e simulação)

Bluetooth: Enquadramento (1/2)

- Norma de tecnologia sem fios de curto alcance, destinada à troca de dados entre dispositivos fixos e móveis a curtas distâncias.
- Opera por ondas rádio nas bandas ISM, entre 2,402 GHz e 2,48 GHz.
- Sustenta a criação de redes de área pessoal (PAN).
- Foi originalmente concebida como alternativa sem fios aos cabos de dados RS-232.
- Aplicações iniciais incluíam a troca de ficheiros entre dispositivos portáteis próximos e a ligação de telemóveis e leitores de música a auscultadores sem fios.

Bluetooth: Enquadramento (2/2)

- Com potência de 2,5 mW, o alcance típico limita-se a cerca de 10 metros.
- A norma é gerida pelo Bluetooth Special Interest Group (Bluetooth SIG).

Bluetooth Clássico (BR/EDR): Rádio e Rede (1/3)

- **Salto de frequência (FHSS):** 79 canais de 1 MHz na banda de 2,4 GHz, cerca de 1600 saltos por segundo, com *slots* de 625 µs. O *Adaptive Frequency Hopping* (AFH, v1.2) evita os canais ocupados por Wi-Fi.
- **Débito no ar:** 1 Mbit/s em BR (GFSK); 2 Mbit/s e 3 Mbit/s em EDR (modulações $\pi/4$-DQPSK e 8DPSK).
- **Piconet:** um mestre e até 7 escravos ativos (endereço de 3 bits); a *scatternet* liga várias piconets através de um dispositivo comum.
- **Ligações:** ACL (dados, comutação de pacotes) e SCO/eSCO (voz, reserva periódica de *slots*); sem estas não há áudio de baixa latência.
- **Descoberta:** *inquiry* (quem está por perto?) seguida de *page* (ligar a um endereço conhecido), operações bem mais pesadas do que o anúncio BLE.

Bluetooth Clássico: Classes de Potência e Perfis (2/3)

Classe	Potência máx.	Alcance típico	Exemplo
1	100 mW (+20 dBm)	$\approx$ 100 m	adaptadores USB, industrial
2	2,5 mW (+4 dBm)	$\approx$ 10 m	telemóveis, auscultadores
3	1 mW (0 dBm)	$\approx$ 1 m	periféricos de curto alcance

- **Perfis** (casos de utilização normalizados): SPP (porta série sobre RFCOMM), HFP/HSP (mãos livres), A2DP/AVRCP (áudio e controlo), HID (teclado e rato), PAN (rede IP).
- **Ligação ao laboratório de IoT:** módulos baratos como o HC-05/HC-06 usam o perfil SPP, ou seja, o "cabo RS-232 sem fios" da origem do Bluetooth; não aparecem num `lescan` BLE.

Evolução das Versões do Bluetooth (3/3)

Versão	Ano	Novidade principal
1.x / 2.0+EDR	1999–2004	BR; EDR (2 e 3 Mbit/s)
2.1	2007	Secure Simple Pairing (SSP)
4.0	2010	**Bluetooth Low Energy**; modo dual
4.1 / 4.2	2013–14	LE Secure Connections (ECDH), pacotes de 251 B, IP sobre BLE (IPSP)
5.0	2016	PHY 2M e Coded, anúncios estendidos (255 B), 8× difusão
5.1	2019	Deteção de direção (AoA/AoD)
5.2	2019	LE Audio (LC3), canais isócronos, EATT
5.3 / 5.4	2021–23	Melhorias de eficiência; PAwR e etiquetas eletrónicas de prateleira
6.0	2024	*Channel Sounding* (medição de distância)

Bluetooth Low Energy: Motivação (1/2)

- Norma de redes sem fios desenvolvida pelo Bluetooth Special Interest Group.
- Constitui um subconjunto leve do Bluetooth clássico, e não uma simples otimização deste.
- Foi acrescentada à norma na versão 4.0 da especificação central do Bluetooth, adotada em 2010.
- Foi concebida para aplicações sem fios de baixo consumo, baixa latência e baixo débito.

Bluetooth Low Energy: Motivação (2/2)

- Não é retrocompatível com dispositivos Bluetooth clássicos: um dispositivo que precise de comunicar com ambos tem de implementar as duas pilhas.
- Daí a distinção entre dispositivos *single mode*, que implementam apenas BLE, e *dual mode*, que suportam simultaneamente BLE e Bluetooth clássico.

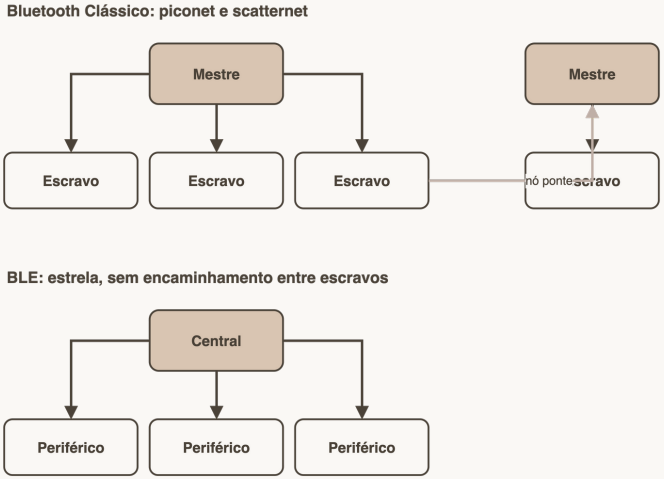
BLE face ao Bluetooth Clássico (1/2)

- **Topologia:** o BLE organiza-se em estrela; o Bluetooth clássico suporta *scatternet*.
- **Débito aplicacional:** cerca de 0,27 Mbit/s no BLE, contra 0,7 a 2,1 Mbit/s no Bluetooth clássico; no ar, 1 Mbit/s contra 1 a 3 Mbit/s.
- **Dimensão da mensagem:** 8 a 47 bytes no BLE, contra 358 bytes no Bluetooth clássico.
- **Consumo:** inferior a 15 mA no BLE, contra menos de 30 mA no Bluetooth clássico.
- **Custo:** baixo no BLE, elevado no Bluetooth clássico.
- **Alcance:** 50 a 150 m no BLE, inferior a 30 m no Bluetooth clássico.

BLE face ao Bluetooth Clássico (2/2)

- **Latência:** 3 ms no BLE, contra cerca de 100 ms no Bluetooth clássico.
- O consumo de referência do Bluetooth clássico ronda 1 W, enquanto o BLE se situa entre 0,01 e 0,5 W consoante o caso de utilização.

Topologias: Clássico e BLE



Papéis dos Dispositivos em BLE (1/2)

- **Dispositivo periférico (servidor):** de pequena dimensão, baixo consumo e recursos limitados, liga-se a um dispositivo central mais capaz. São exemplos os monitores de ritmo cardíaco e as etiquetas de proximidade BLE.
- **Dispositivo central (cliente):** dispõe de maior capacidade de processamento e de memória. São exemplos os telemóveis e os tablets.

Papéis dos Dispositivos em BLE (2/2)

- Esta assimetria é deliberada: concentra a complexidade no dispositivo com energia disponível e mantém o sensor simples e económico.
- Três pares de papéis distintos não devem confundir-se, por operarem em camadas diferentes: mestre e escravo, na camada de ligação; central e periférico, ao nível do GAP; cliente e servidor, ao nível do GATT.

Camada Física do BLE (1/2)

- Opera na banda ISM dos 2,4 GHz.
- Débito de 1 Mbit/s, com modulação GFSK.
- O índice de modulação é superior ao do Bluetooth de débito básico, o que se traduz em melhor alcance.
- O espectro está dividido em 40 canais, com espaçamento de 2 MHz.

Camada Física do BLE (2/2)

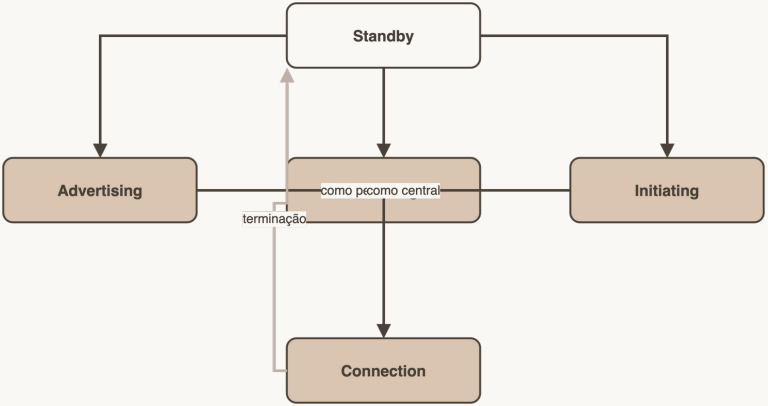
- Três desses canais (o 37, o 38 e o 39) são reservados para anúncios; os restantes, numerados de 0 a 36, transportam dados durante as ligações estabelecidas.
- A separação entre canais de anúncio e canais de dados permite que a descoberta de dispositivos não interfira com as ligações já em curso.

Bluetooth 5: Novas Camadas Físicas

PHY	Débito	Alcance relativo	Nota
LE 1M	1 Mbit/s	$1\times$	obrigatório; BLE 4.x
LE 2M	2 Mbit/s	$\approx 1\times$ (menor margem)	metade do tempo no ar por bit
LE Coded S=2	500 kbit/s	$\approx 2\times$	codificação FEC
LE Coded S=8	125 kbit/s	$\approx 4\times$	máximo alcance, menor débito

- Os **anúncios estendidos** usam os canais de dados como canais secundários e transportam até 255 bytes por pacote (contra 31 bytes nos anúncios legados).
- Trocar débito por alcance é o mesmo compromisso das redes LPWAN da Aula 03, agora à escala de dezenas a centenas de metros.
- Um telemóvel ou um adaptador sem suporte a 5.0 não vê anúncios estendidos: convém verificar a versão no `hciconfig -a`.

Máquina de Estados da Camada de Ligação



Camada de Ligação: Estados (1/2)

- **Standby:** o dispositivo não transmite nem recebe dados, e não está ligado a nenhum outro dispositivo.
- **Advertiser:** difunde periodicamente pacotes de anúncio.
- **Scanner:** procura ativamente dispositivos em estado de anúncio.
- **Initiator:** tenta ativamente iniciar uma ligação com outro dispositivo.
- **Master:** encontra-se ligado a outro dispositivo, no papel de mestre.

Camada de Ligação: Estados (2/2)

- **Slave:** encontra-se ligado a outro dispositivo, no papel de escravo.
- A transição entre estes seis estados descreve todo o ciclo de vida de uma comunicação BLE, da descoberta à terminação da ligação.

Eventos de Anúncio (1/2)

- Durante um evento de anúncio, é transmitido um pacote em cada um dos três canais de anúncio: 37, 38 e 39.
- A repetição nos três canais aumenta a probabilidade de o anúncio ser recebido, mesmo que um dos canais esteja sujeito a interferência.
- Um dispositivo em estado de anúncio transmite pacotes que podem transportar uma carga útil de dados.

Eventos de Anúncio (2/2)

- Os anúncios podem ser dirigidos a um dispositivo específico ou não dirigidos, isto é, destinados a qualquer recetor.
- Os anúncios podem ainda ser conectáveis ou não conectáveis; neste último caso, servem apenas para difusão de dados, sem intenção de estabelecer ligação.

Tipos de Anúncio e Varrimento (1/2)

- **Conectável não dirigido:** qualquer dispositivo em varrimento pode iniciar uma ligação com este anunciante.
- **Conectável dirigido:** apenas um dispositivo específico pode iniciar a ligação.
- **Não conectável não dirigido:** nenhum dispositivo pode iniciar ligação; usa-se sobretudo para difusão geral de dados.
- **Descobrível não dirigido:** qualquer dispositivo em varrimento pode pedir mais informação ao anunciante, mas não pode estabelecer ligação.

Tipos de Anúncio e Varrimento (2/2)

- **Varrimento passivo:** o dispositivo escuta os canais de anúncio e, ao receber um pacote, encaminha a informação para as camadas superiores.
- **Varrimento ativo:** o dispositivo responde ao anúncio com um pedido de varrimento, ao qual o anunciante responde com informação adicional.

Estrutura do Pacote de Anúncio (1/2)

- A carga útil legada tem até **31 bytes**, organizada em estruturas *AD*: comprimento, tipo e valor. A resposta a varrimento acrescenta mais 31 bytes.
- Tipos frequentes: Flags (`0x01`), UUID de serviços (`0x02`/`0x03`), nome completo (`0x09`), potência de emissão (`0x0A`), dados do fabricante (`0xFF`).
- Exemplo real de um sensor: `02 01 06 | 0D 09 54 43 49 6F 54 2D 53 65 6E 73 6F 72`, isto é, Flags = `0x06` (descobrível, sem BR/EDR) e nome "TCIoT-Sensor".
- **Porque só alguns dispositivos mostram nome?** O nome pode estar apenas na resposta a varrimento, que só chega a quem faz varrimento *ativo*.
- Com 31 bytes não há lugar para tudo: o programador escolhe entre nome, UUID ou dados, e remete o resto para a resposta ou para o GATT.

Balizas (Beacons) e Estimativa de Distância (2/2)

- Uma **baliza** é um anunciante não conectável que difunde um identificador: *iBeacon* (Apple: UUID, *major*, *minor*, potência a 1 m) e *Eddystone* (Google, UUID de serviço `0xFEAA`).
- O recetor mede o **RSSI** e estima a distância com o modelo log-distância: $d = 10^{(P_{1\,m} - \mathrm{RSSI})/(10\,n)}$, com n entre 2 (espaço livre) e 4 (interior).
- Exemplo: $P_{1\,m} = -59$ dBm, RSSI $= -75$ dBm, $n = 2{,}5 \Rightarrow d \approx 4{,}4$ m.
- O RSSI varia vários dB por reflexões e pelo corpo humano: serve para zonas (perto, longe), não para metros exatos.
- O 5.1 (AoA/AoD) e o 6.0 (*Channel Sounding*) existem precisamente para ultrapassar este limite.

Estabelecimento de Ligação e Parâmetros (1/2)

- Um dispositivo em varrimento passa a iniciador quando envia ao anunciante um pacote de pedido de ligação, que transporta o conjunto de parâmetros da camada de ligação destinados ao futuro escravo.
- **Channel Map:** indica quais os canais de dados utilizados durante a ligação.
- **Hop Increment:** valor aleatório entre 5 e 16, usado pelo algoritmo de seleção de canal.
- **Connection Interval:** múltiplo de 1,25 ms, entre 7,5 ms e 4,0 s.

Estabelecimento de Ligação e Parâmetros (2/2)

- **Supervision Timeout:** múltiplo de 10 ms, entre 100 ms e 32,0 s.
- **Slave Latency:** valor entre 0 e 499.

Eventos de Ligação e Latência do Escravo (1/2)

- Os eventos de ligação ocorrem periodicamente, com o período definido pelo parâmetro *connection interval*.
- Cada evento decorre num canal de dados, entre o 0 e o 36, sendo o canal seguinte determinado pelo parâmetro *hop increment*.
- Os eventos ocorrem mesmo quando nenhuma das partes tem dados a enviar, o que garante a manutenção da ligação mas consome energia.
- A *slave latency* define o número máximo de eventos de ligação que o escravo pode ignorar.

Eventos de Ligação e Latência do Escravo (2/2)

- Este mecanismo permite ao escravo saltar eventos quando não tem dados a transmitir, reduzindo o consumo sem terminar a ligação.
- Se o escravo não responder durante um evento, o mestre reenvia o pacote nos eventos seguintes até obter resposta.

Parâmetros de Ligação e Seu Efeito



Compromissos na Escolha dos Parâmetros (1/2)

- **Intervalo de ligação curto:** consumo mais elevado, débito superior e menor espera até ao envio dos dados.
- **Intervalo de ligação longo:** consumo reduzido, débito inferior e maior espera até ao envio dos dados.
- **Latência do escravo nula ou baixa:** consumo mais elevado, com receção de dados mais imediata.
- **Latência do escravo elevada:** consumo reduzido, com receção de dados menos imediata.

Compromissos na Escolha dos Parâmetros (2/2)

- Se o escravo considerar os parâmetros inadequados, pode enviar ao mestre um pedido de atualização da ligação, propondo um intervalo dentro de uma gama pretendida.
- Estes parâmetros são, na prática, o principal instrumento de que o programador dispõe para equilibrar autonomia e capacidade de resposta.

Orçamento Energético: Ordem de Grandeza

- Corrente média num ciclo: $I_{med} = \dfrac{I_{ev}\, t_{ev} + I_{rep}\,(T - t_{ev})}{T}$, com T o intervalo de anúncio ou de ligação.
- Exemplo: anúncio a 1 s, evento de 1 ms a 8 mA, repouso a 2 µA $\Rightarrow$ $I_{med} \approx 10$ µA.
- Pilha CR2032 (≈ 220 mAh): $220\,\text{mAh}/0{,}010\,\text{mA} \approx 22\,000$ h, cerca de 2,5 anos (na prática menos: autodescarga e picos de corrente).
- Passar o intervalo para 100 ms multiplica a corrente média por cerca de 10; em contrapartida a descoberta passa a ser quase imediata.
- O Bluetooth clássico, com ligação permanente na ordem das dezenas de mA, não admite este perfil de funcionamento.

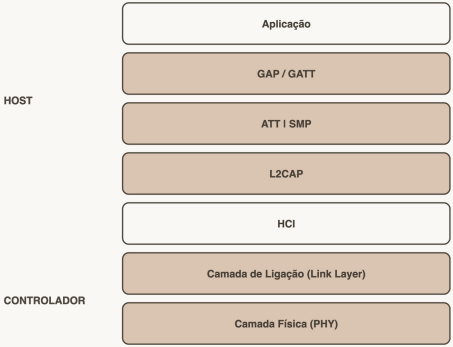
Terminação da Ligação (1/2)

- A ligação pode ser terminada voluntariamente por qualquer das partes, mestre ou escravo, por qualquer motivo.
- Pode ainda terminar por esgotamento do temporizador de supervisão.
- Quando o temporizador atinge o valor de *supervision timeout*, o dispositivo considera a ligação perdida.

Terminação da Ligação (2/2)

- Nesse caso, sai do estado de ligação e regressa ao estado de anúncio, de varrimento ou de repouso, consoante o seu papel.
- Este mecanismo garante que um dispositivo não permanece indefinidamente à espera de um par que deixou de responder.

Pilha Protocolar do BLE



O HCI separa o host (tipicamente software) do controlador (rádio)

Pilha Protocolar: HCI e L2CAP (1/2)

- A pilha BLE divide-se em duas secções principais: o *controller*, que trata a rádio e a camada de ligação, e o *host*, que suporta os protocolos superiores.
- A **Host Controller Interface (HCI)** é a fronteira entre as duas: não realiza processamento próprio, servindo como interface através da qual o host comunica com as camadas inferiores.

Pilha Protocolar: HCI e L2CAP (2/2)

- O **Logical Link Control and Adaptation Protocol (L2CAP)** permite que os protocolos e aplicações de nível superior transmitam e recebam pacotes de dados de dimensão limitada (a MTU por omissão do ATT é de 23 bytes, como se detalha adiante).
- O L2CAP assegura ainda a gestão de canais, permitindo canais lógicos entre dois extremos, suportados pela camada de ligação.

MTU, Débito Útil e UUID

- O **ATT_MTU** por omissão é de **23 bytes**; retirando 3 bytes de cabeçalho ATT, cada leitura ou notificação transporta 20 bytes úteis. É este o valor de 23 bytes a ter presente nas aplicações.
- A MTU negoceia-se após a ligação (até 517 bytes) e, desde o 4.2, o pacote da camada de ligação pode chegar a 251 bytes (*Data Length Extension*).
- Os **UUID** de 16 bits são abreviaturas do UUID base do SIG: `0x180F` ≡ `0000180F-0000-1000-8000-00805F9B34FB`. É esta forma longa que o `gatttool` mostra.
- Valores com mais de um byte seguem *little-endian*: `2E 09` = `0x092E` = 2350, isto é 23,50 °C em centésimos.

Topologia de Rede (1/2)

- O BLE organiza-se numa topologia em estrela: o dispositivo mestre gere a ligação e pode estar ligado a vários escravos.
- Cada escravo só pode estar ligado a um mestre de cada vez dentro da mesma ligação.
- Cada escravo comunica com o mestre num canal físico distinto.
- Um dispositivo pode manter ligações físicas com mais do que um mestre em simultâneo, e pode inclusivamente desempenhar os papéis de mestre e de escravo ao mesmo tempo.

Topologia de Rede (2/2)

- Daqui resultam as designações de *piconet*, para o conjunto formado por um mestre e os seus escravos, e de *scatternet*, para a interligação de várias *piconets*.
- Numa rede BLE ligada, dois periféricos não comunicam diretamente: todas as mensagens passam pelo dispositivo central.

Generic Access Profile (GAP)

- Controla os processos de ligação e de anúncio, e é o que torna um dispositivo visível aos restantes.
- Determina como dois dispositivos podem, ou não podem, interagir entre si, e define o papel de cada um.
- Define um perfil base que todos os dispositivos Bluetooth implementam, articulando as várias camadas para formar os requisitos mínimos de um dispositivo Bluetooth.
- Define ainda procedimentos genéricos para serviços associados à ligação: descoberta de dispositivos, estabelecimento, gestão e terminação da ligação, e iniciação das funcionalidades de segurança.

GAP: Papéis e Modos de Descoberta (1/2)

- **Broadcaster:** anunciante não conectável, que apenas difunde informação.
- **Observer:** procura anúncios, mas não pode iniciar ligações.
- **Peripheral:** anunciante conectável, que opera como escravo numa única ligação da camada de ligação.
- **Central:** procura anúncios e inicia ligações, operando como mestre numa ou em várias ligações.
- **Modo não descobrível:** o dispositivo não emite anúncios.

GAP: Papéis e Modos de Descoberta (2/2)

- **Modo de descoberta limitada:** o dispositivo anuncia-se durante um período limitado, findo o qual regressa ao estado de repouso.
- **Modo de descoberta geral:** o dispositivo anuncia-se continuamente.
- O GAP gere ainda os dados enviados nos pacotes de anúncio e nas respostas a pedidos de varrimento.

Segurança: Security Manager Protocol (1/2)

- O Security Manager Protocol (SMP) realiza a autenticação e a gestão de chaves, usando AES-128 como algoritmo de cifra.
- Trabalha em articulação com o GAP para gerir as relações entre dispositivos, através de três mecanismos distintos.
- **Emparelhamento (*pairing*):** estabelece cifra entre dois dispositivos depois de a ligação ter sido criada.

Segurança: Security Manager Protocol (2/2)

- **Autenticação:** verifica que o dispositivo par é de confiança, protegendo contra ataques de intermediário (*man-in-the-middle*).
- **Vinculação (*bonding*):** estabelece uma relação de longo prazo entre dispositivos, com armazenamento da informação de segurança e identidade.

Emparelhamento: Chaves e Capacidades (1/2)

- O emparelhamento pode ser iniciado pelo dispositivo central ou pelo periférico.
- Os dois dispositivos geram e trocam chaves de curto prazo (*short-term keys*), usadas para decifrar os pacotes de dados.
- Qualquer dos dispositivos pode pedir a ativação da vinculação, criando uma relação de longo prazo.

Emparelhamento: Chaves e Capacidades (2/2)

- Nesse caso é gerada, trocada e armazenada uma chave de longo prazo (*long-term key*), que permite voltar a cifrar a ligação rapidamente numa reconexão, sem repetir todo o processo de emparelhamento.
- Cada dispositivo declara as suas capacidades de entrada e saída: **DisplayOnly**, sem entrada mas com saída de dados; **DisplayYesNo**, com entrada limitada a sim ou não e capacidade de mostrar dados; **KeyboardOnly**, com introdução de palavra-passe mas sem visor; **NoInputNoOutput**, sem qualquer interação com o utilizador; e **KeyboardDisplay**, com ambas as capacidades.

Métodos de Emparelhamento (1/2)

- A combinação das capacidades de entrada e saída dos dois dispositivos determina qual dos métodos de emparelhamento é utilizado.
- **Introdução de chave (*passkey entry*):** um dispositivo apresenta uma chave gerada aleatoriamente, e o outro exige que o utilizador a introduza. Este método produz uma ligação autenticada, com proteção contra ataques de intermediário.

Métodos de Emparelhamento (2/2)

- **"Just Works":** o processo conclui-se sem introdução de qualquer chave. A ligação é cifrada, mas não autenticada, o que a deixa vulnerável a ataques de intermediário.
- Se um dos dispositivos não tiver capacidade de entrada nem de saída, o método "Just Works" é o único possível, limitação com implicações de segurança relevantes em sensores simples.

Segurança: Evolução e Vulnerabilidades

Mecanismo	Como funciona	Fraqueza conhecida
LE Legacy (4.0/4.1)	chave temporária (TK) trocada; *Just Works*: TK = 0	um *sniffer* passivo recupera a chave de longo prazo (ferramenta Crackle)
LE Secure Connections (4.2)	ECDH em P-256; comparação numérica	*Just Works* continua sem autenticação contra intermediário
Classic SSP (2.1)	ECDH, comparação numérica	KNOB (2019) reduz a entropia da chave; BIAS (2020) permite personificação
Pilhas e SoC	—	BlueBorne (2017), SweynTooth (2020): falhas de implementação

– Boas práticas: exigir LE Secure Connections, autenticar leituras sensíveis, e usar endereços privados resolúveis.

Attribute Protocol (ATT) (1/2)

- Um atributo é um valor discreto ao qual estão associadas três propriedades: um identificador (*handle*), que funciona como endereço; um tipo; e um conjunto de permissões.
- O ATT define o protocolo usado no ar para ler, escrever e descobrir atributos.
- A arquitetura é do tipo cliente-servidor: os servidores detêm os dados e expõem-nos sob a forma de atributos; os clientes pretendem utilizá-los.

Attribute Protocol (ATT) (2/2)

- O papel de cliente ou servidor de um dispositivo é independente do papel central ou periférico no GAP, e do papel mestre ou escravo na camada de ligação.
- Numa tabela de atributos, o *handle* indica o endereço do atributo, o tipo indica o que os dados representam (através de um UUID atribuído pelo Bluetooth SIG ou de um tipo personalizado) e as permissões determinam se e como o cliente pode aceder ao valor.

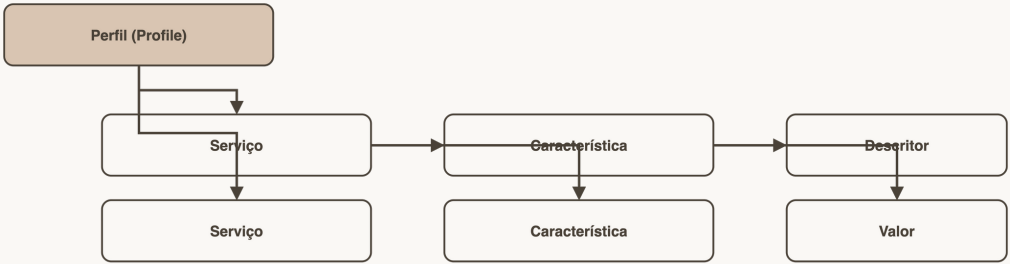
Generic Attribute Profile (GATT) (1/2)

- Define como dois dispositivos BLE trocam dados, através dos conceitos de serviço e de característica.
- Só entra em funcionamento depois de estabelecida uma ligação dedicada entre os dois dispositivos, e a comunicação é bidirecional.
- Estabelece uma relação cliente-servidor: o periférico é o servidor GATT, que detém as definições de serviços e características; o central é o cliente GATT, que envia pedidos ao servidor.
- **Perfil:** coleção predefinida de serviços, compilada pelo Bluetooth SIG ou pelo projetista do periférico.

Generic Attribute Profile (GATT) (2/2)

- **Serviço:** coleção de características, identificada por um UUID único, de 16 a 128 bits.
- **Característica:** encapsula um único ponto de dados. Como é possível escrever numa característica, este é também o mecanismo para enviar dados de volta ao periférico.

Hierarquia de Atributos GATT



Cada atributo tem um identificador (UUID), um handle e permissões de acesso

GATT: Transações e Hierarquia (1/2)

- Todas as transações são iniciadas pelo cliente GATT, que recebe do servidor a respetiva resposta.
- Ao estabelecer a ligação, o periférico sugere um intervalo de ligação; o dispositivo central tenta contactá-lo em cada intervalo para verificar a existência de novos dados.
- Um perfil é composto por um ou mais serviços necessários para satisfazer um caso de utilização concreto.
- Um serviço pode conter atributos designados valores de característica, que são os valores utilizados pelo serviço.

GATT: Transações e Hierarquia (2/2)

- Cada valor de característica é obrigatoriamente precedido de um atributo de declaração de característica, que descreve as suas propriedades.
- As características podem ainda incluir atributos descritores opcionais, com informação adicional sobre o seu conteúdo ou configuração.

GATT: Procedimentos do Cliente (1/2)

- O GATT define os procedimentos de utilização do protocolo de atributos para descobrir, ler, escrever e obter indicações sobre atributos.
- Define também o agrupamento e a relação entre características dentro de um serviço ou perfil, e os procedimentos de configuração da difusão de atributos.
- **Descobrir característica por UUID:** procura no servidor todos os atributos cujo tipo corresponda ao UUID indicado.
- **Ler valor de característica:** lê o valor da característica no *handle* especificado.

GATT: Procedimentos do Cliente (2/2)

- **Escrever valor de característica:** escreve um novo valor na característica indicada.
- Estes procedimentos constituem a base prática da programação BLE, retomada na aula seguinte.

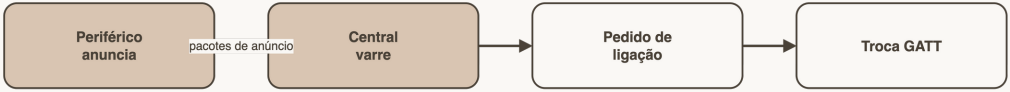
Notificações e Indicações (CCCD) (1/2)

- Em vez de o cliente repetir leituras (*polling*), o servidor pode **notificar** quando o valor muda, poupando energia e rádio.
- A subscrição faz-se escrevendo no descritor **CCCD** (UUID `0x2902`) da característica: `0x0001` ativa notificações, `0x0002` indicações, `0x0000` desativa.
- **Notificação:** sem confirmação. **Indicação:** o cliente confirma cada mensagem (mais fiável, mais lenta).
- No `gatttool`: `char-write-req <handle do CCCD> 0100` e, depois, linhas `Notification handle = ... value: ....`
- O CCCD só existe nas características com a propriedade *notify* ou *indicate*; o seu *handle* segue-se ao do valor.

Exemplo: Serviço de Bateria (2/2)

- Serviço `0x180F`; característica Battery Level `0x2A19`: um byte, de 0 a 100 (%).
- Leitura: `char-read-hnd 0x0008` $\Rightarrow$ 57; em decimal, $0\text{x}57 = 87$, ou seja, 87 % de carga.
- Serviços normalizados de uso corrente: Generic Access `0x1800`, Device Information `0x180A`, Heart Rate `0x180D`, Environmental Sensing `0x181A`.
- Um UUID de 128 bits aleatório indica serviço do fabricante: o significado dos bytes só se conhece pela documentação, ou por engenharia inversa.
- Regra prática: interpretar sempre *handle* $\rightarrow$ UUID $\rightarrow$ propriedades $\rightarrow$ formato do valor.

Difusão por Anúncio e Troca por Ligação



Difusão sem ligação (beacon) termina no primeiro passo; a troca de dados estruturados exige ligação

Comunicação por Anúncio e por Ligação (1/2)

- O BLE suporta dois paradigmas de comunicação distintos, e a escolha entre eles condiciona toda a arquitetura da aplicação.
- **Comunicação baseada em anúncio:** sem ligação estabelecida, do tipo difusão, unidirecional. O periférico emite dados que qualquer dispositivo na vizinhança pode receber.
- **Comunicação baseada em ligação:** orientada à ligação, bidirecional, sustentada pelo GATT. Exige o estabelecimento prévio de uma ligação dedicada.

Comunicação por Anúncio e por Ligação (2/2)

- A difusão dispensa a negociação de ligação e é mais económica em energia, mas não oferece confirmação de receção nem confidencialidade.
- A comunicação por ligação permite leitura, escrita e notificações, ao custo de manter a ligação ativa.

Aplicações Típicas (1/2)

- O BLE foi concebido para consumo muito reduzido e ciclos de trabalho baixos, o que define o seu campo de aplicação.
- **Objetos que transportamos:** relógios, acessórios de saúde e desporto, sensores corporais.
- **Objetos que nos rodeiam:** etiquetas de localização, etiquetas usadas como chaves, sensores e atuadores de domótica.
- **Comunicação dentro de sistemas:** automação de escritórios, automação industrial e comunicação no interior de veículos.

Aplicações Típicas (2/2)

- **Perfil de proximidade:** define o comportamento quando um dispositivo se afasta do seu par, ao ponto de a ligação cair ou de a atenuação do sinal ultrapassar um limiar predefinido, gerando um alerta imediato.
- **Serviço de perda de ligação:** recorre à característica de nível de alerta para sinalizar no dispositivo a perda da ligação.

Endereços BLE e Privacidade

Tipo	Característica
Público	atribuído pelo IEEE, fixo: rastreável para sempre
Aleatório estático	gerado no arranque, fixo até reiniciar
Privado resolúvel (RPA)	muda cada $\approx$ 15 min; só quem possui a chave IRK (após *bonding*) o reconhece
Privado não resolúvel	aleatório e temporário; para balizas

- Quase todos os telemóveis anunciam-se com RPA: por isso o endereço visto num `lescan` muda entre varrimentos e é um endereço **aleatório**.
- Consequência prática: ligar com o `gatttool` exige a opção `-t random`.
- Se a pulseira do caso de estudo usasse RPA, o rastreio por anúncios deixaria de ser trivial; falhas de implementação ainda o permitem em alguns produtos.

Caso de Estudo: Pulseiras de Atividade (1/2)

- As pulseiras de atividade ligam-se ao telemóvel apenas quando a aplicação correspondente está em execução em primeiro plano.
- Enquanto a aplicação corre, a pulseira comunica ativamente e sincroniza os dados de atividade, como número de passos e calorias.
- Quando a aplicação deixa de correr, a ligação é terminada e a pulseira regressa ao estado de anúncio.
- Neste estado, o dispositivo continua a emitir pacotes de anúncio, que qualquer recetor na vizinhança pode captar.

Caso de Estudo: Pulseiras de Atividade (2/2)

- Daqui resulta uma fuga de privacidade: a emissão contínua de anúncios permite rastrear a presença e a deslocação do utilizador, mesmo sem qualquer ligação estabelecida.
- Este exemplo ilustra que as decisões de projeto ao nível do protocolo têm consequências diretas em privacidade.

Ferramentas de Gestão em Linux (1/2)

- **hciconfig:** lista os adaptadores BLE ligados ao sistema. Com `hciconfig hciX up` e `hciconfig hciX down` ativa-se e desativa-se o adaptador indicado.
- **hcitool:** configura ligações Bluetooth e envia comandos específicos aos dispositivos. O comando `hcitool lescan` procura dispositivos BLE na vizinhança.
- **gatttool:** permite descobrir, ler e escrever características. Executado com a opção `-I`, abre um modo interativo onde se podem emitir comandos como `connect <endereço>`.

Ferramentas de Gestão em Linux (2/2)

- Estas ferramentas serão utilizadas nas aulas práticas, quando o Raspberry Pi assumir o papel de cliente BLE.
- Do lado das aplicações móveis, o nRF Connect e o BLE Scanner permitem inspecionar serviços e características de dispositivos próximos.

Ferramentas Modernas do BlueZ

Clássica (deprecated)	Equivalente atual
`hciconfig`	`bluetoothctl list`, `show`; `btmgmt info`
`hcitool lescan`	`bluetoothctl scan on` (mostra RSSI, TxPower, dados do anúncio)
`gatttool -I`	`bluetoothctl`: `connect`, `menu gatt`, `list-attributes`, `select-attribute`, `read`, `notify on`
`hcidump`	`btmon` (traço HCI em tempo real e gravação em ficheiro)

- O BlueZ marca `hciconfig`, `hcitool`, `gatttool`, `hcidump`, `rfcomm` e `sdptool` como *deprecated*. O Raspberry Pi OS Bookworm (BlueZ 5.66) ainda os inclui, mas noutras distribuições podem faltar.
- Em Python, a biblioteca `bleak` funciona em Linux, macOS e Windows e é o caminho natural para o código das Aulas 05 e 06.

Sem Hardware: Emulação, Simulação e Análise Offline

Opção	Para quê
`ble_sim.py` (guião)	simulador de consola que imita `hciconfig`, `hcitool lescan` e `gatttool`; sem dependências
Wireshark + ficheiro `btsnoop`	análise offline de anúncios, ATT e GATT (filtros `btle`, `btatt`); o mesmo ficheiro do *HCI snoop log* do Android
Bumble (Python)	pilha Bluetooth com controladores virtuais: dois dispositivos virtuais comunicam sem rádio (`pip install bumble`)
nRF Connect (Android/iOS)	o telemóvel emula um periférico com GATT configurável, útil também sem Raspberry Pi

– Nada disto substitui o rádio real (interferência, RSSI, alcance), mas permite exercitar o raciocínio sobre serviços, características, notificações e anúncios.

Captura de Tráfego para Análise

- O Android dispõe, a partir da versão 4.4, de uma opção para registar todos os pacotes Bluetooth que entram e saem do dispositivo.
- A ativação faz-se em dois passos: ativar as opções de programador nas definições do sistema e, dentro destas, ativar o registo *Bluetooth HCI snoop log*.
- O ficheiro gerado pode depois ser analisado com ferramentas de análise de protocolos, permitindo observar a sequência real de anúncios, ligações e transações GATT.
- Esta capacidade é útil tanto no desenvolvimento, para diagnosticar problemas de comunicação, como na análise de segurança de dispositivos BLE.

Síntese da Aula (1/2)

- O Bluetooth clássico e o BLE partilham a marca e a banda de operação, mas são tecnologias distintas e não interoperáveis: o BLE troca débito e dimensão de mensagem por consumo e latência muito inferiores.
- A camada de ligação organiza toda a comunicação em torno de seis estados e de dois modos: anúncio, para difusão, e ligação, para troca bidirecional.
- Os parâmetros de ligação (intervalo, latência do escravo e temporizador de supervisão) são o principal instrumento para equilibrar autonomia e capacidade de resposta.

Síntese da Aula (2/2)

- O GAP define visibilidade e papéis; o ATT define o formato dos dados expostos; o GATT define como cliente e servidor os trocam através de serviços e características.
- A segurança assenta no emparelhamento, na autenticação e na vinculação, mas as capacidades de entrada e saída limitadas dos sensores empurram muitas implementações para o método "Just Works", sem proteção contra intermediários.
- A aula seguinte aplica estes conceitos na implementação de um servidor BLE em Arduino.

Fontes

Gonçalves, P. *Apresent4 — Bluetooth*, TCIoT, edições anteriores. Base de toda a exposição sobre camada física, camada de ligação, GAP, GATT, emparelhamento e ferramentas de gestão em Linux.

Townsend, K. *et al.* (2014). *Getting Started with Bluetooth Low Energy*. O'Reilly. Bluetooth SIG, *Core Specification* e *Assigned Numbers*; documentação do BlueZ e do Bumble (`google.github.io/bumble`).

Bluetooth SIG. *Bluetooth Core Specification*. Definição normativa de GAP, GATT, ATT, L2CAP e SMP.

Collotta, M. *et al.* (2018). Bluetooth 5: A Concrete Step Forward toward the IoT. *IEEE Communications Magazine*. Disponível em `Leituras/`.

Jadhav, S. *et al.* (2016). Bluetooth 5 and Internet of Things: Potential and architecture. *SCOPES*. Disponível em `Leituras/`.

Obrigada.

Questões e comentários